

Towards Agile Testing for Railway Safety-critical Software *

Nan Li

School of Information Science
and Technology
Southwest Jiaotong University
Chengdu, Sichuan, China
nli1@gmu.edu

Jin Guo

School of Information Science
and Technology
Southwest Jiaotong University
Chengdu, Sichuan, China
jguo_scce@home.swjtu.edu.cn

Jeff Lei

Department of Computer
Science and Engineering
The University of Texas at
Arlington
Arlington, TX, USA
ylei@cse.uta.edu

Yao Li

School of Information Science
and Technology
Southwest Jiaotong University
Chengdu, Sichuan, China
ly.9967@163.com

Chang Rao

School of Information Science
and Technology
Southwest Jiaotong University
Chengdu, Sichuan, China
bk20103050@my.swjtu.edu.cn

YaXin Cao

School of Information Science
and Technology
Southwest Jiaotong University
Chengdu, Sichuan, China
997385024@qq.com

ABSTRACT

EN 50128 gives a general guidance for testing railway safety-critical software. However, it does not specify how to test safety-critical software in an agile development environment. Based on our observation, agile software development has not been applied to the development of the Chinese Train Control System (CTCS). Previous research focused on agile methods for managing and planning safety-critical software development. There exists little research about agile testing for developing railway safety-critical software in the literature.

In this paper, we identify several major shortcomings of the existing practice of testing railway safety-critical software in China. In particular, we find out, that Continuous Integration (CI) and Continuous Delivery (CD) are missing from the current practice of agile testing. Furthermore, effective techniques and tools are lacking for CTCS. To address these shortcomings, we propose an agile testing framework for developing railway safety-critical software. In this framework, a build automation tool manages source code, unit tests, integration tests, resources and other tools. This test framework supports CI and CD, making agile testing possible. In addition, it includes effective unit and integration testing tools that allow rigorous tests to be generated. We believe this test framework will not only be applied to railway software, but also used for other safety-critical software.

*This work is funded by the China Railway Corporation under Grant 2015X009-D.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

XP '16 May 24–27, 2016, Edinburgh, UK

© 2016 ACM. ISBN 978-1-4503-2138-9.

DOI: 10.1145/2962695.2962713

Keywords

Agile Testing; Railway Safety-critical System; Continuous Integration; Continuous Delivery; Unit Testing; Integration Testing

1. INTRODUCTION AND PROBLEMS

Rigorous testing is essential to ensure the quality of safety-critical software. Like other types of software such as web applications and web services, developing safety-critical software may also face the challenges of frequent changes. Changes in safety-critical software are often due to changes in detailed system design and faults detected by testing, instead of changes in requirements. Thus, it is important to test safety-critical software in an agile manner. Agile testing requires software practitioners to handle various changes quickly using agile testing tools, without compromising safety and quality. EN 50128 [8] provides a generic guidance for testing railway safety-critical software. However, it does not provide best practices of agile testing from the general software industry yet.

When testing the Chinese Train Control System (CTCS) on high-speed railways in an ongoing project, we find three general problems. First, *Continuous Integration (CI)* and *Continuous Delivery (CD)* have not been applied in practice. CI is an extremely important agile development practice in which every code commit is pulled automatically from a source code repository and verified automatically by an automated build. CD ensures every change can be reliably released at any time in agile development after a successful CI execution. Please note that CD is not equal to *Continuous Deployment*, which automatically deploys software to production. In CD, the software is verified that it can be deployed but is not deployed. CI and CD create a fast feedback loop. If an error occurs during an automated build, developers and testers would make immediate notice and correction. The major obstacle that prevents practitioners from adopting CI and CD is that many railway software development and testing tools are not compatible with each other. Second, unit tests are primarily generated by hand.

Practitioners have few tools available to generate tests that satisfy advanced code coverage such as *Modified Condition, Decision Coverage (MCD)*. Third, integration testing is not performed adequately. In the software industry, the distinction among integration testing, system testing, and user acceptance testing often blurs. We use integration testing in this paper to emphasize that different components must be well integrated to meet system specifications. Existing approaches use simple scenarios that are manually generated to test CTCS. Systematic coverage criteria need to be satisfied to test railway safety-critical software.

The first problem is directly related to agile software development since CI and CD are the core concepts of agile testing. The second and third problems focus more on test generation and coverage. However, these two problems are also related because test generation tools need to be compatible with the CI and CD framework. Since the focus of this paper is agile testing, we will emphasize how to integrate unit testing and integration testing into CI and CD, instead of a detailed discussion on test generation and coverage techniques.

In this paper, we propose an agile testing framework for railway safety-critical software. This framework provides a high-level architecture of agile testing that supports CI, CD, unit testing, and integration testing. To the best of our knowledge, little research has been reported about agile testing for railway safety-critical software. Ge et al. [10] proposed an iterative development approach for the general safety-critical software domain. Myklebust et al. [15] proposed the *SafeScrum* model for railway safety-critical software development, from the software project management perspective.

The rest of the paper is organized as follows. Section 2 presents the agile testing framework and talks about several problems of agile testing and their general solutions. Section 3 summarizes the conclusions in agile testing for railway safety-critical software and discusses the future work.

2. AGILE TESTING

Figure 1 shows the proposed agile testing framework. On the right side, the big box represents the System Under Test (SUT). SUT has to be stored on a source code repository such as *GitHub* [11]. SUT usually includes source code, related resources and dependencies, unit tests, and integration tests, which is managed by a build automation tool. This build tool manages all the related tools used for SUT including testing tools. During development, when a code change is committed, CI is triggered to start a build that includes compilation of the source code and executions of unit and integration tests. When the build passes, SUT can be proved to be deployable in CD. Unlike general-purpose software such as web applications and web services, deploying railway software automatically and directly to railway stations and trains is not realistic. Even in CD, automatically verifying software on railway is not practical. Thus, to support CD, SUT should be automatically verified on a simulation platform for testing.

Next, we will discuss the details of CI in Section 2.1. In Sections 2.2 and 2.3, we will talk about concrete problems in unit and integration testing and how the agile testing framework helps us address these problems.

2.1 Continuous Integration

We find that researchers in the safety-critical software development field have not paid enough attention to the fundamental concept of agile development, i.e., Continuous Integration (CI). Specifically, CI requires that every agile development tool is compatible with the build tool. As shown in Figure 1, the build tool manages source code, tests, and all tools including test frameworks, code coverage tools, static analysis tools, etc. For example, in Java programming, a build automation tool, *Apache Maven* [9] could manage a unit testing framework, *JUnit* [2], a coverage measurement tool, *JaCoCo* [12], and a static analysis tool *CheckStyle* [3]. When a change of requirement, design, or test happens, developers implement the change and commit the code into a source code repository. The new commit is pulled from the source code repository by a CI tool. The CI tool removes the project dependencies that are established in the development environment and runs the build tool for a completely new build. Thus, CI ensures SUT is not developed in a specific environment or on a specific machine. Then the test frameworks, code coverage tools, and static analysis tools are executed against the new commit in the new build. CI has been widely adopted in developing general-purpose software. For example, *Jenkins* [13] and *Travis* [6] have been widely used in agile software development.

When we test CTCS, we find that the development tools are not compatible with each other. Specifically, *Microsoft Visual Studio* has been used to develop *C/C++* programs for CTCS. However, since *Microsoft Visual Studio* is proprietary, it only supports *Microsoft* products. Other tools such as *Safety-Critical Application Development Environment (SCADE) Suite* from *Esterel Technologies* are not compatible with *Microsoft Visual Studio*. Therefore, when developing railway safety-critical software, we need to build a CI environment in which every tool is compatible with each other.

2.2 Unit Testing

Safety-critical software is no more complex than general-purpose software in terms of the lines of code (LOC). As reported by Durelli et al. [7], LOC of three safety-critical software used in CTCS ranges from 42,380 to 65,887, while some of open source general-purpose software can easily go above 500,000 such as *Apache Hadoop*. We believe that agile testing techniques and tools such as statics analysis and code coverage tool used for general-purpose software can be applied to railway safety-critical software. Then what is different for safety-critical software?

We believe safety-critical software requires more rigorous testing and uses stronger test coverage criteria in unit testing. Using stronger test coverage does not conflict with the agile development philosophy. Instead, better testing tools need to be built to help practitioners generate tests to satisfy various coverage criteria. Durelli et al. [7] compared general-purpose non-safety-critical software with safety-critical software used in CTCS in terms of the number of predicates and the number of clauses in every predicate. They reported that safety-critical programs have more complex predicates than non-safety-critical programs. Specifically, 4.83% of the predicates in the safety-critical programs have four or more clauses while the non-safety-critical programs in the same size range include only 0.54% predicate with at least four clauses.

When testing predicates for general-purpose software, testers

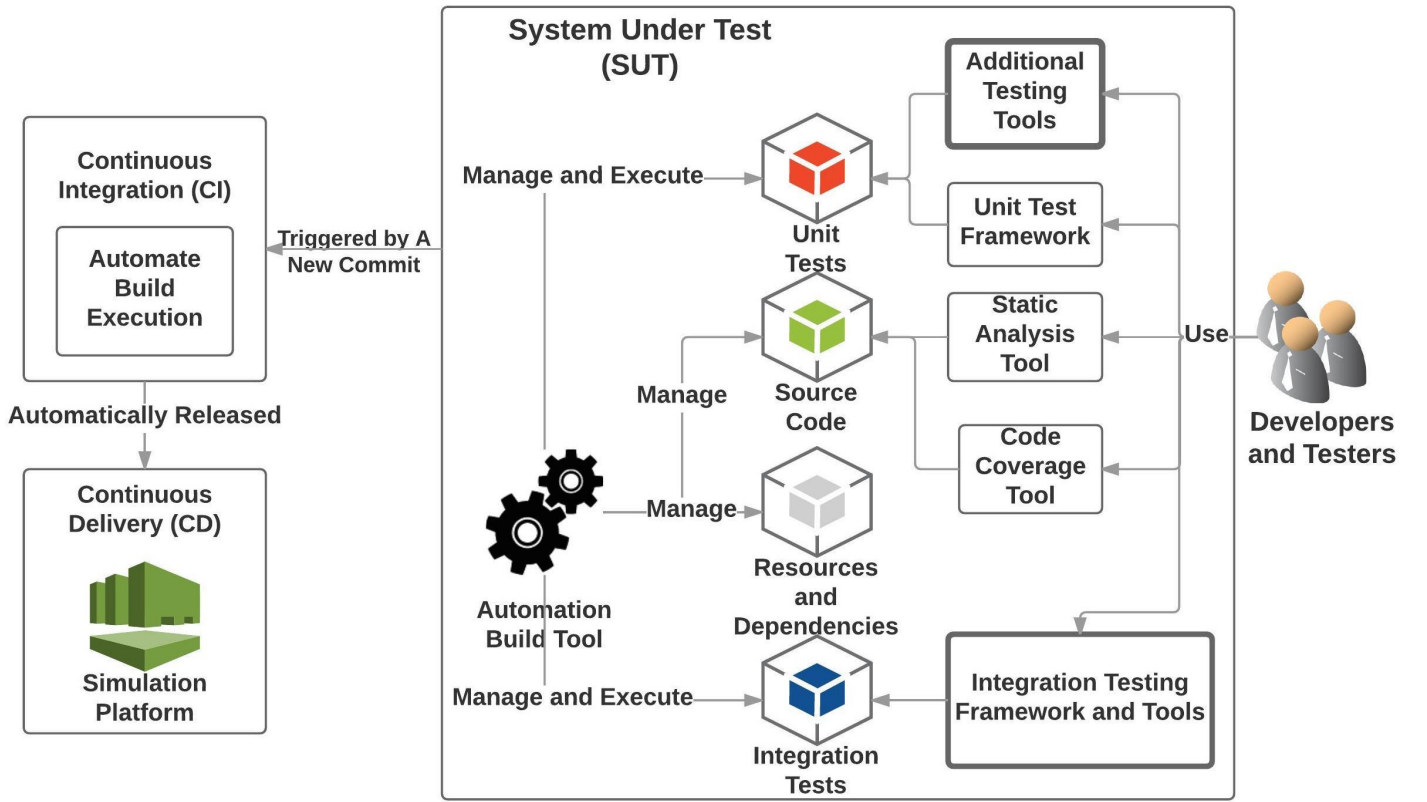


Figure 1: Agile Testing Framework

usually use branch coverage, which is also called predicate coverage in logic testing [1]. Predicate coverage requires every predicate to be evaluated to true and false at least once. For testing predicates in safety-critical software, testers should use strong coverage such as *Modified Condition, Decision Coverage (MCDC)*. The US Federal Aviation Administration (FAA) explicitly requires MCDC [4] to certify safety critical parts of avionics software in commercial aircraft [16]. Likewise, railway external certification bodies also use MCDC to inspect railway safety-critical software. So having tests to cover MCDC in agile unit testing is necessary. In agile development of safety-critical software, complex predicates may be changed frequently. Thus testers may have to frequently update tests to satisfy MCDC. Because there are a lot more complex predicates in safety-critical software than non-safety-critical software and satisfying MCDC requires much more tests than predicate coverage, logic testing may require a lot of time and efforts in agile testing for safety-critical software. To quickly satisfy MCDC when changes occur, practitioners need effective logic testing tools to generate tests and measure MCDC automatically. Moreover, such logic testing tools should be compatible with existing build and CI tools such that the logic testing tools can be used in agile testing.

2.3 Integration Testing

A simulation platform is currently used to perform integration testing of CTCs. This simulation platform includes simulated stations, rails, signals, blocks, etc. During integration testing, several relatively simple test scenarios are

manually generated and executed against SUT on this simulation platform. This integration testing practice can be improved in the following two aspects. First, the simulation platform can be improved to provide different types of stations. Second, advanced test coverage can be employed to generate more comprehensive test scenarios. In this paper, we focus on the second aspect.

In an ongoing research project that studies integration testing of CTCs, we have used model-based testing and combinatorial testing to generate tests. This combined integration testing technique can be used for both software developers and external certification bodies. In Model-Based Testing (MBT), we generate test paths (abstract tests) to achieve some well-defined graph coverage criteria from a behavioral model such as UML state machine diagram. A behavioral model describes behaviors of SUT. Each test path includes transitions and constraints such as state invariants, pre-conditions, and post-conditions. Each transition consists of various input parameters from Track Circuit (TC) or other components. When the source code mapped to a transition is executed, actual values of outputs are evaluated against constraints that must be satisfied by expected outputs. Because a test path may include many transitions, there may be many input parameters in a test path. Therefore, if each parameter has multiple test values, we may generate too many concrete (and executable) tests when we use all combinations of parameter values. Testing every single combination of input parameters is not practical and in some case not possible, even for safety-critical software.

In our project, we have used combinatorial testing to re-

duce the number of tests. Combinatorial testing has been shown to be a very effective software testing strategy. The key observation is that most faults are caused by interactions involving only a few parameters [14]. A t-way combinatorial test set, or simply t-way test, is designed to cover all the t-way interactions, i.e., interactions involving no more than t input parameters. When input parameters are properly modeled, a t-way test set is guaranteed to detect faults that are caused by interactions involving no more than t parameters [5].

We will not present the detailed integration testing solution since the focus of the paper is on agile testing. Again, when we use MBT and combinatorial testing tools to generate integration tests, we must ensure the tools are compatible with the agile CI and CD philosophy.

How to generate effective integration tests to verify requirements for assurance is challenging. This step requires transformation from human-readable documents to machine-readable formats, test automation, test coverage, and test generation. Satisfying requirements makes the test generation complicated, but it should not affect CI and CD after tests are generated. Thus, further discussion on this step is also out of scope of this paper.

3. CONCLUSIONS AND FUTURE WORK

In this paper, we discuss several problems about agile testing that are found when we test the Chinese Train Control System (CTCS), including no support for Continuous Integration (CI) and Continuous Delivery (CD) environments and lack of effective unit and integration testing. We present an agile testing framework to address these problems. In this framework, build automation tools, software development tools (IDEs), test frameworks, code coverage measurement tools, and static analysis tools must be compatible with each other to support CI and CD. Potentially, other tools such as impact analysis tools can be added to the framework, with compatibility with CI and CD. In addition, the framework includes test generation tools to satisfy MCDC at the unit testing level and use advanced techniques such as MBT and combinatorial testing at the integration testing level. We believe this agile testing framework could also be applied to the development of other types of safety-critical software.

In the future, we will need to find out a concrete solution in terms of tools to use to solve the incompatibility problem so that we could build CI and CD environments for CTCS.

4. ADDITIONAL AUTHORS

Additional authors: Xiuxuan Wang (School of Information Science and Technology, Southwest Jiaotong University, Chengdu, Sichuan, China, email: wxx016005@163.com), Jian Wang (School of Information Science and Technology, Southwest Jiaotong University, Chengdu, Sichuan, China, email: 649040337@qq.com), Zengshu Shi (School of Information Science and Technology, Southwest Jiaotong University, Chengdu, Sichuan, China, email: shizengshu@126.com), and Xinge Ai (School of Information Science and Technology, Southwest Jiaotong University, Chengdu, Sichuan, China, email: axg1967@sina.com).

5. REFERENCES

- [1] P. Ammann and J. Offutt. *Introduction to Software Testing*. Cambridge University Press, Cambridge, UK, 2008.
- [2] K. Beck, E. Gamma, D. Staff, and M. Clark. JUnit. Online, 2002. <http://junit.org>, last access Feb 2016.
- [3] O. Burn. Checkstyle. Online, 2001. <http://checkstyle.sourceforge.net/>, last access Dec 2015.
- [4] J. J. Chilenski and S. P. Miller. Applicability of Modified Condition/Decision Coverage to Software Testing. *Software Engineering Journal*, 9(5):193–200, 1994.
- [5] D. M. Cohen, S. R. Dalal, M. L. Fredman, and G. C. Patton. The AETG system: An approach to testing based on combinatorial design. *IEEE Transactions on Software Engineering*, 23(7):437–444, July 1997.
- [6] T. C. community. Travis-CI. Online, 2011. <https://travis-ci.org/>, last access Dec 2015.
- [7] V. H. Durelli, J. Offutt, N. Li, M. E. Delamaro, J. Guo, Z. Shi, and X. Ai. What to expect of predicates: An empirical analysis of predicates in real world programs. *Journal of Systems and Software*, 113:324 – 336, 2016.
- [8] EN50128:2011. Railway applications – software for railway control and protections systems. Technical report, European Committee for Electrotechnical Standardization, 2011.
- [9] A. S. Foundation. Apache Maven. Online, 2004. <https://maven.apache.org/>, last access Dec 2015.
- [10] X. Ge, R. F. Paige, and J. A. McDermid. An iterative approach for development of safety-critical software and safety arguments. In *Agile Conference (AGILE)*, 2010, pages 35–43, Aug 2010.
- [11] Github. Github. Online, 2007. <http://github.com>, last access Feb 2016.
- [12] M. Hoffmann, B. Janiczak, E. Mandrikov, and M. Friedenhagen. Jacoco code coverage tool. Online, 2009. <http://eclemma.org/jacoco/>, last access Dec 2015.
- [13] K. Kawaguchi. Jenkins. Online, 2011. <https://jenkins-ci.org/>, last access Jan 2016.
- [14] D. R. Kuhn, D. R. Wallace, and J. A. M. Gallo. Software fault interactions and implications for software testing. *IEEE Transactions on Software Engineering*, 30(6):418–421, June 2004.
- [15] T. Myklebust, T. Stålhane, and N. Lynbgy. Application of an agile development process for EN 50128/railway conformant software. In *Safety and Reliability of Complex Engineered Systems: ESREL 2015*, pages 84–89, Zurich, Switzerland, September 2015.
- [16] RTCA-DO-178B. Software Considerations in Airborne Systems and Equipment Certification, December 1992.

[1] P. Ammann and J. Offutt. *Introduction to Software Testing*. Cambridge University Press, Cambridge, UK,